



Universidad Autónoma de Yucatan

Facultad De Matematicas



Fecha: **05 de Junio de 2026**

Asignatura: **Programacion Orientada a Objetos**

Actividad:

Proyecto Final – Gestor de Tareas

Reporte

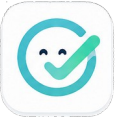
Profesor: **Canul Chin Miguel Angel**

Equipo: **Dream Team**

Integrantes:

Gutierrez Lopez Miguel Angel

Lomas Lopez Leonardo



Introduccion:

El siguiente documento tecnico describe la funcionalidad, estructura y diseño del programa-sistema "Gestion de Tareas", codificado en java para la clase de Programacion Orientado a Objetos.

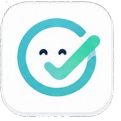
Se describira los siguientes aspectos claves:

1. **Proposito.**
2. **Funcionalidad.**
3. **Diagrama de Clases.**
4. **Metodologia.**
5. **Resultados y Analisis.**
6. **Conclusiones.**

Reporte de Proyecto Final : Dream Team

Proyecto: **Gestor de Actividades**





Proposito del programa

El sistema **Gestor de Actividades** es un programa diseñado para ayudar al alumnado que lo requiera en su administracion de sus actividades escolares.

Este sistema cuenta con tres tipos de actividades que el usuario puede administrar, estas son:

- Proyectos: Un proyecto contiene una serie de tareas(objetivos) que se cumplen para completar el proyecto
- Tarea: La actividad basica
- Examen: n/a

Estas actividades enlistadas, son componentes de las Materias, el sistema cuenta con una gestion de estas actividades por Materia, pudiendo saber a que Materia, pertenece cada Actividad.

Funcionalidad

A continuacion se describira la funcionalidad del sistema enlistada por funciones que la componen:

Clase App: Es la clase con el metodo Main, encargada unicamente de inicializar el menu, y desplegarlo en terminal al usuario

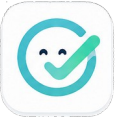
Clase Abstracta Actividad: Describe los atributos de forma generalizada que debe de tener cada una de las actividades (Tarea, Proyecto, Examen).

Clase ControladorArchivo: Es la clase encargada de controlar la lectura y escritura del archivo que se usa para guardar los datos del sistema, este tiene en si dos metodos que se encargan de cumplir estos dos aspectos mencionados anteriormente:

- leerMaterias: Metodo que cumple el labor de obtener el archivo donde se guardaron los datos y leerlo de principio a fin, a modo de obtener todos los datos y poder suministrarlos al menu donde se usaran para su despliegue al usuario.
- EscribirMaterias: Metodo que cumple el labor de sobrescribir los datos del archivo con los nuevos proporcionados por el usuario.

Clase Examen Hereda de **Actividad**: Representa al tipo Examen del Sistema, este es identico a la **Tarea**, se dejo asi para hacerlo mas legible en tiempo de diseño.

Interfaz GestionActividad: Encargada de definir los metodos que deben de implementar las clases **Proyecto** y **Materia**, para el correcto



Gestor de Tareas – Programa

agregado y eliminado de las Actividades correspondientes

Clase Materia implementa **GestionActividad**:

Como su nombre lo indica, esta clase es la Materia del sistema, dentro de la cual destaca el uso de una lista que contiene todas las actividades asociadas a esa materia.

Clase Proyecto Hereda de **Actividad** implementa **GestionActividad**: Clase que representa los proyectos del sistema, esta clase contiene un atributo de tipo lista con el cual accede a las tareas asociadas al proyecto.

Clase Tarea Hereda de **Actividad**: Clase que representa las Tareas del sistema.

Clase Menu: Clase principal que tiene dentro de si toda la logica que el usuario vera por terminal. Involucra varios metodos los cuales se describiran en breve:

- **Menu()**: Constructor para inicializar el objeto
- **principal()**: Menu con las opciones principales para navegar por el sistema, iniciando desde las materias existentes en el sistema.
- **menuMateria()**: Le muestra al usuario para administrar las Actividades de esa Materia y su posible edicion de la Materia
- **menuTareas()**: Muestra las tareas de la materia seleccionada
- **menuTareas()**: Metodo sobrecargado para mostrar las tareas pertenecientes a un proyecto.
- **menuProyectos()**: Menu para administrar los proyectos de la materia seleccionada.
- **menuExamenes()**: Menu para administrar los examenes de la materia seleccionada
- **menuAgregarMateria()**: Menu para agregar materias, recabando sus datos de la misma.
- **menuAgregarActividad()**: Menu para agregar una tarea a un proyecto.
- **menuAgregarActividad()**: Menu para agregar nuevas Actividades a la materia seleccionada.
- **menuActividad()**: Menu que permite administrar la Actividad seleccionada.
- **menuActividad()**: Sobrecargado, menu para administrar las actividades de un proyecto asociado (se eligio asi por diseño)
- **menuEditarMateria()**: Menu para editar los atributos de la Materia seleccionada
- **menuEditarActividad()**: Menu para editar los atributos de una actividad seleccionada.
- **LeerEntero()**: metodo que sirve para obtener un numero puesto por el usuario por terminal.
- **menuSalir()**: Menu para salir del programa, es necesario para guardar todos los cambios hechos por el usuario.

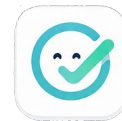


Diagrama de Clases

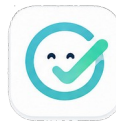
El diagrama de clases, debido a su tamaño y resolución, se optó por dejarlo como un archivo png aparte para su consulta.

Metodologia

Después de una investigación (algo exhaustiva) concluimos que queríamos crear un sistema que pudiera ayudar en algo simple pero importante **la gestión de tareas**, quedamos de acuerdo en realizar este proyecto, debido a ello nos pusimos a trabajar, ese trabajo se describirá más adelante como parte de la metodología de trabajo que usamos:

Administración del proyecto: Se optó por usar la plataforma GitHub para poder ir trabajando asincrónicamente en el proyecto, esto nos permitió un avance rápido.

Diseño del Sistema: Optamos por un diseño simple que pudiera distribuir las responsabilidades de todo el sistema entre todas las clases, usando los principios SOLID. Este diseño enfoca las funcionalidades de archivos a una clase única, la funcionalidad del despliegue de menús a una clase en concreto y las clases subsiguientes cumplen roles similares.



Resultados y Analisis

Al terminar el proyecto pudimos obtener un aprendizaje profundo acerca de los conceptos fundamentales que le otorgan al paradigma de la programacion orientada a objetos de su reconocimiento, diferenciando lo de la programacion estructurada.

Se pudo obtener un resultado decente del sistema propuesto al inicio, pudiendo completarse, aunque para ello se tuvo que recortar la implementacion de la interfaz grafica y la funcionalidad de las notificaciones, mas que nada por el hecho de temas mas complejos que involucraba el poder llevarlas a cabo, junto con la cuestion de tiempo.

Se adjuntara en esta seccion capturas de pantalla del funcionamiento del programa para verificar que se obtuvo el resultado deseado.

```
Escoga la opcion de su conveniencia (#numerica)
Materias disponibles:
1.- P00
Opciones
2.- Agregar una Materia
3.- Salir del programa
Selecion:
```

Conclusiones

Al concluir el proyecto, este quedo en su mayoría como lo habiamos propuesto desde un inicio, resaltando el primer diagrama de clases que habiamos propuesto.

Al terminar este proyecto pudimos recabar un conocimiento mas profundo de este nuevo paradigma, y a su vez pudimos mejorar nuestra logica y habilidades con la programacion, resaltando el hecho de ver las entidades del sistema como objetos separas, tal como el paradigma lo indica, y tambien pudimos hacer uso de sus ventajas, tales como la herencia para la reutilizacion de codigo, el polimorfismo y la abstraccion y el encapsulamiento.

```
Materia: P00
Profesor: Martin Guevara

1.- Proyectos
2.- Tareas
3.- Exámenes
4.- Editar materia
5.- Eliminar materia
6.- Regresar
Seleccion: 2
1.- Investigacion Documental

Seleccione una tarea o una opcion de las siguientes
2.- Agregar una nueva tarea
3.- Regresar
Seleccion: 
```